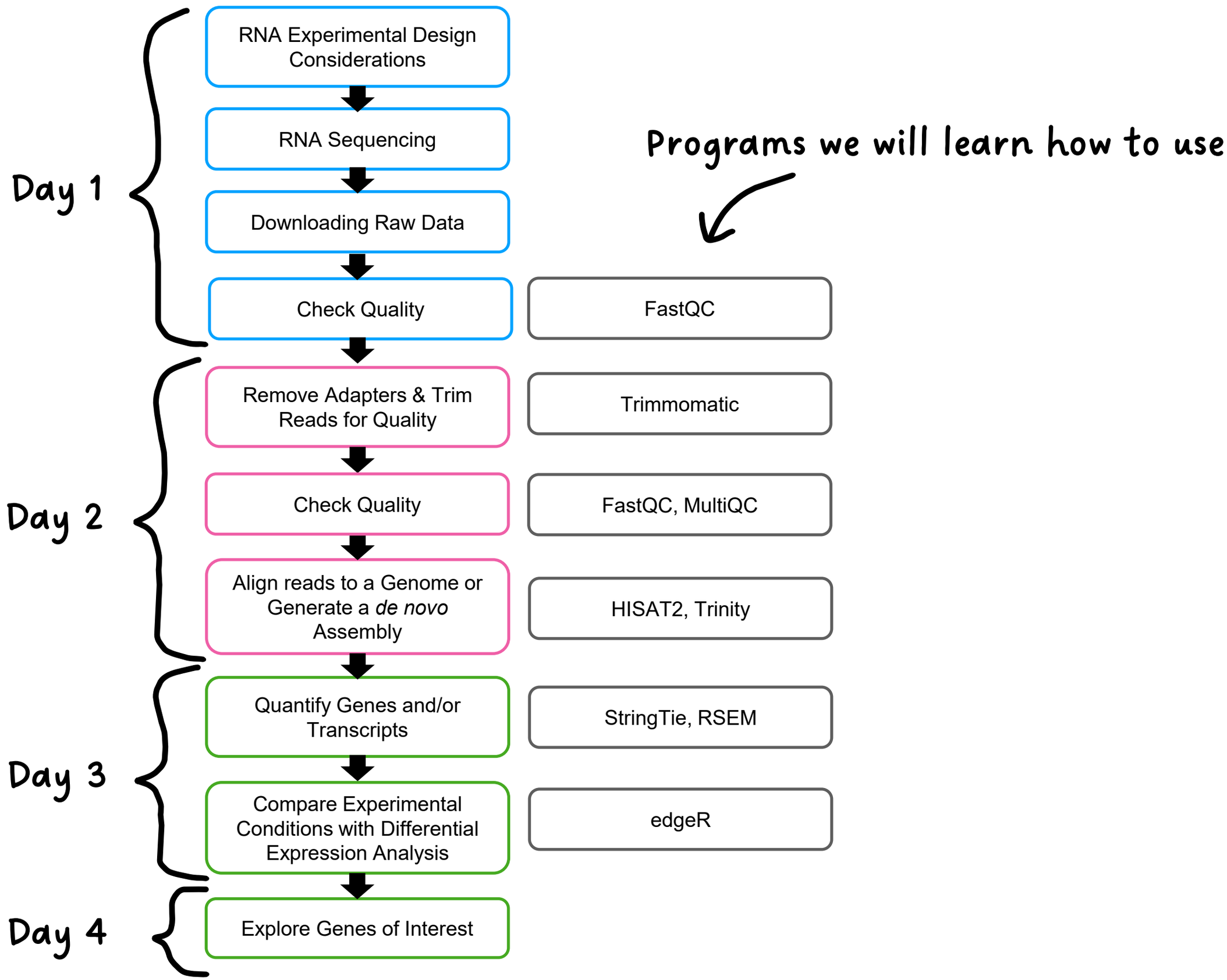
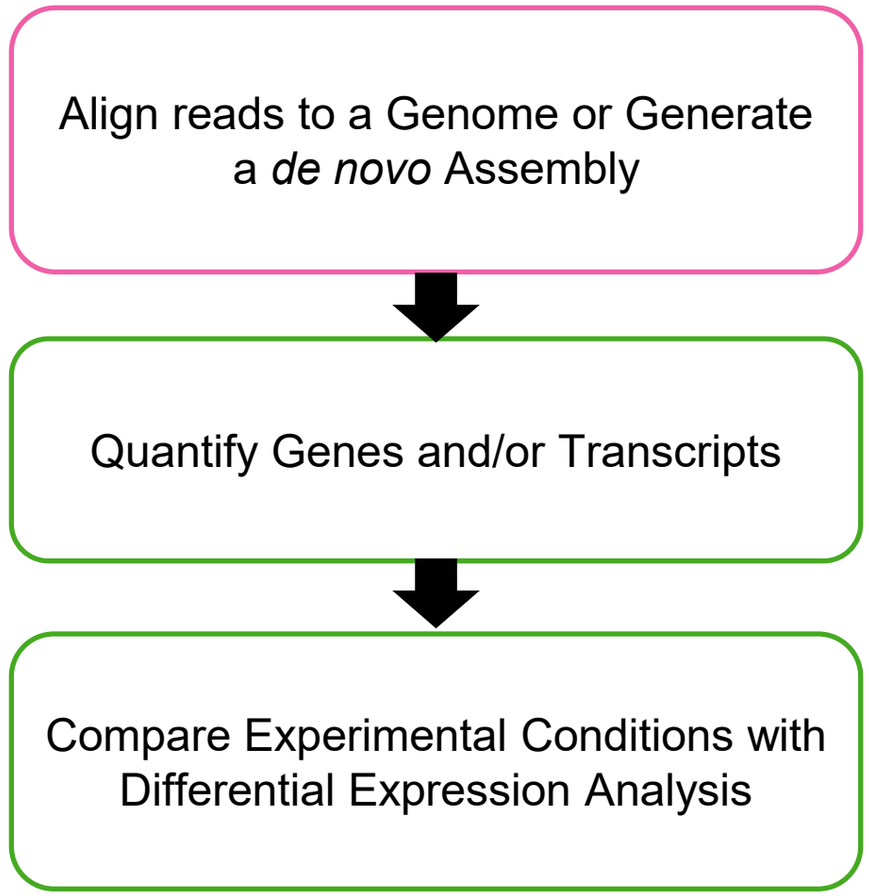
RNA-seq and analysis of differential gene expression

Slides adapted from: Tom Booker & Bernadette Johnson
Bioinformatics for Evolutionary Biology and Conservation

Basic RNA-seq pipeline



Basic RNA-seq pipeline



Let's check our files from last time

Did everything run according to plan?

1. Did we trim and filter our reads?

Should have paired and unpaired for R1 and R2.

```
ubuntu@bioinformatics-1-do-not-delete:~$ ls 02_trimreads/
SRR19736866_R1_paired.fastq    SRR19736866_R2_paired.fastq    logs
SRR19736866_R1_unpaired.fastq  SRR19736866_R2_unpaired.fastq
```

2. If something went wrong, we should be able to check our log file.

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat 02_trimreads/logs/*
TrimmomaticPE: Started with arguments:
 /home/ubuntu/01_fastqfiles/SRR19736866_1.fastq /home/ubuntu/01_fastqfiles/SRR19736866_2.fastq /ho
me/ubuntu/02_trimreads/SRR19736866_R1_paired.fastq /home/ubuntu/02_trimreads/SRR19736866_R1_unpair
ed.fastq /home/ubuntu/02_trimreads/SRR19736866_R2_paired.fastq /home/ubuntu/02_trimreads/SRR197368
66_R2_unpaired.fastq ILLUMINACLIP:/home/ubuntu/Trimmomatic-0.36/adapters/TruSeq3-PE-2.fa:2:30:10:2
:keepBothReads LEADING:3 TRAILING:3 MINLEN:75
Using PrefixPair: 'TACACTCTTTCCCTACACGACGCTCTTCCGATCT' and 'GTGACTGGAGTTCAGACGTGTGCTCTTCCGATCT'
Using Long Clipping Sequence: 'AGATCGGAAGAGCGTCGTGTAGGGAAAGAGTGTA'
Using Long Clipping Sequence: 'AGATCGGAAGAGCACACGTCTGAACTCCAGTCAC'
Using Long Clipping Sequence: 'GTGACTGGAGTTCAGACGTGTGCTCTTCCGATCT'
Using Long Clipping Sequence: 'TACACTCTTTCCCTACACGACGCTCTTCCGATCT'
ILLUMINACLIP: Using 1 prefix pairs, 4 forward/reverse sequences, 0 forward only sequences, 0 rever
se only sequences
Quality encoding detected as phred33
Input Read Pairs: 17081609 Both Surviving: 11893076 (69.63%) Forward Only Surviving: 4562609 (26.7
1%) Reverse Only Surviving: 186114 (1.09%) Dropped: 439810 (2.57%)
TrimmomaticPE: Completed successfully
```

Things to check in the log report:

- Were Illumina adapters trimmed?
- How many reads survived trimming (and pairing) "Both Surviving"?
- Do you see "Completed successfully"?

Let's check our files from last time

Did everything run according to plan?

1. Did we generate a HISAT2 index?

Should have a bunch of files for a single indexed genome

```
ubuntu@bioinformatics-1-do-not-delete:~/03_hisat2$ ls -l hisat2_index/
total 644612
-rwxrw-r-- 1 ubuntu ubuntu 147070299 Mar  4 03:53 genome_index.1.ht2
-rwxrw-r-- 1 ubuntu ubuntu 107100176 Mar  4 03:53 genome_index.2.ht2
-rwxrw-r-- 1 ubuntu ubuntu      4562 Mar  4 03:43 genome_index.3.ht2
-rwxrw-r-- 1 ubuntu ubuntu 107100171 Mar  4 03:43 genome_index.4.ht2
-rwxrw-r-- 1 ubuntu ubuntu 189781431 Mar  4 03:54 genome_index.5.ht2
-rwxrw-r-- 1 ubuntu ubuntu 108994004 Mar  4 03:54 genome_index.6.ht2
-rwxrw-r-- 1 ubuntu ubuntu        12 Mar  4 03:43 genome_index.7.ht2
-rwxrw-r-- 1 ubuntu ubuntu         8 Mar  4 03:43 genome_index.8.ht2
```

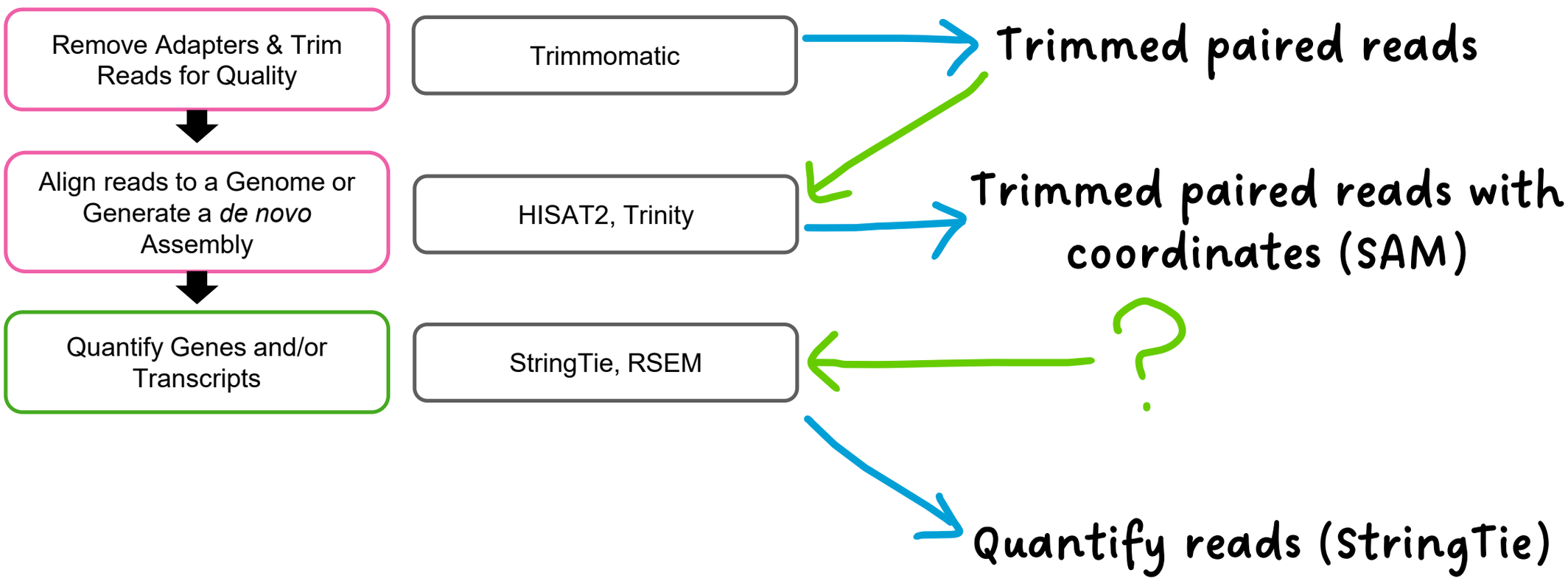
2. Did we run HISAT2 to align our reads (per sample) to the index?

```
ubuntu@bioinformatics-1-do-not-delete:~/03_hisat2$ ls
SRR19736866.sam  hisat2_index  log
ubuntu@bioinformatics-1-do-not-delete:~/03_hisat2$ cat log/SRR19736866_log.txt
11893076 reads; of these:
  11893076 (100.00%) were paired; of these:
    1682474 (14.15%) aligned concordantly 0 times
    9935759 (83.54%) aligned concordantly exactly 1 time
    274843 (2.31%) aligned concordantly >1 times
    ----
    1682474 pairs aligned concordantly 0 times; of these:
      41900 (2.49%) aligned discordantly 1 time
    ----
    1640574 pairs aligned 0 times concordantly or discordantly; of these:
      3281148 mates make up the pairs; of these:
        2466083 (75.16%) aligned 0 times
        795995 (24.26%) aligned exactly 1 time
        19070 (0.58%) aligned >1 times
89.63% overall alignment rate
```

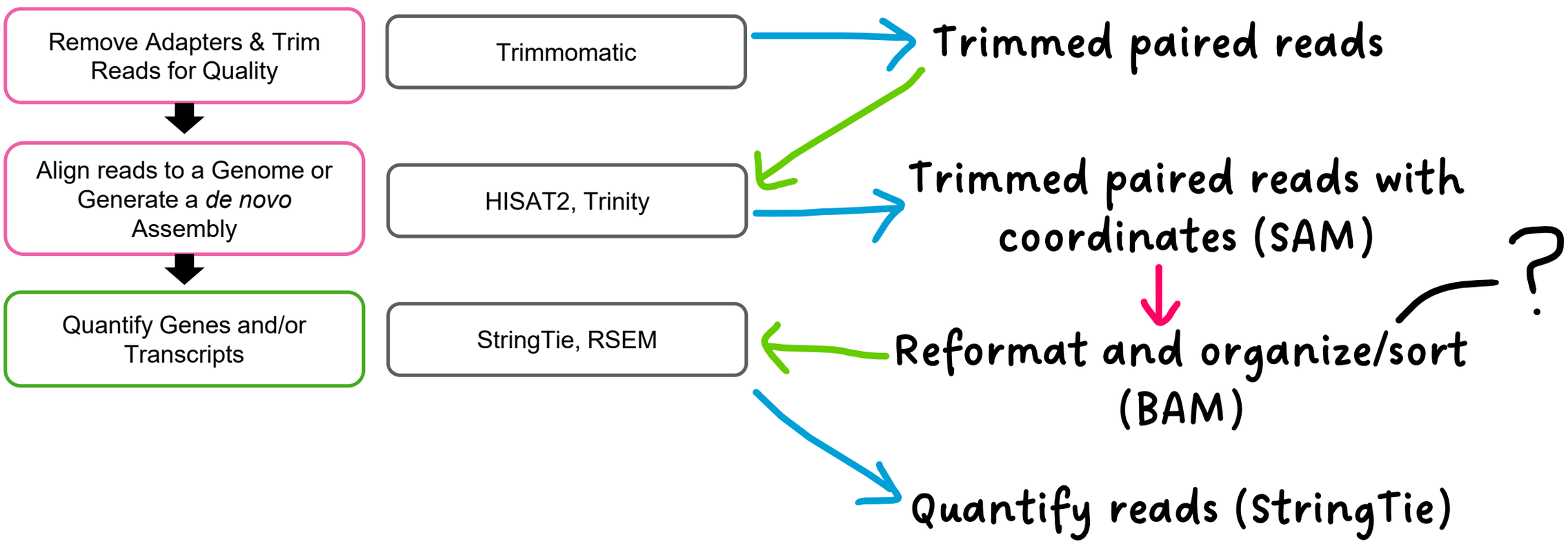
We should have 1 SAM file for each sample.

We should also have 1 log for each sample that tells us the alignment rate for our sample. Ideally you want this to be over ~70%.

Pipeline Recap



Pipeline Recap



Running Samtools

1. We start by converting the SAM into a BAM. This is compressing the file, making it smaller and easier to read.

> samtools view

SAM files are uncompressed, human-readable files

```
ubuntu@bioinformatics-3-do-not-delete:/data/workshop/03_hisat2sam$ head C4NPP.sam
@HD     VN:1.0  SO:unsorted
@SQ     SN:NC_090847.1  LN:30098933
@SQ     SN:NC_090848.1  LN:28880413
@SQ     SN:NC_090849.1  LN:25446114
@SQ     SN:NC_090850.1  LN:24172726
@SQ     SN:NC_090851.1  LN:21913520
@SQ     SN:NC_090852.1  LN:20411112
@SQ     SN:NC_090853.1  LN:20118474
@SQ     SN:NC_090854.1  LN:19128117
@SQ     SN:NC_090855.1  LN:17757856
```

BAM files are compressed, binary versions of the SAM that make it easier and quicker to process

```
ubuntu@bioinformatics-1-do-not-delete:~/04_sortedbam$ head -n 5 C4NPP.bam
```

Running Samtools

1. We can start by converting the sam into a bam file format. This is compressing the file, making it smaller and easier to read.

> samtools view

2. Then we can sort the bam by coordinates. The file is currently in read order (read 1, read 2, read 3…) but we want to sort by genomic position to make it faster for StringTie to count.

> samtools sort

Since we need to run these commands on a bunch of samples, we need to design a more efficient script!

Running Samtools

Store paths as variables

Make output folder

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"
```

Running Samtools

Store paths as variables

Make output folder

For all the files in the input directory, with a .sam extension, do...

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

1. Give the basename of a file (don't include the .sam extension)

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

2. Output on the terminal this phrase

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

3. Run samtools view to convert sam into bam

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

4. Run samtools sort to organize alignments in bam by coordinates

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

After running these four commands, you are done and can loop to the next file.

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

Store paths as variables

Make output folder

Once you loop through all the samples, and run these commands, print "All done" and the date.

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

Let's build a for loop! This helps us run a command on a series of files (when we have multiple samples)!

Running Samtools

```
ubuntu@bioinformatics-1-do-not-delete:~$ cat run_samtools.sh
#!/bin/bash

# Define directories
INPUT_DIR="/home/ubuntu/03_hisat2sam"
OUTPUT_DIR="/home/ubuntu/04_sortedbam"

# Create output and log directories if they don't exist
mkdir -p "$OUTPUT_DIR"

# Loop through all SAM files in the input directory
for SAM in "$INPUT_DIR"/*.sam
do
    # Extract the filename without path or extension
        BASENAME=$(basename "$SAM" .sam)

    # Tell me which sample is running
        echo "Processing $BASENAME..."

    # Convert SAM to BAM
        samtools view -bS "$SAM" > "$OUTPUT_DIR/$BASENAME.bam"

    # Sort BAM file
        samtools sort "$OUTPUT_DIR/$BASENAME.bam" -o "$OUTPUT_DIR/$BASENAME.sorted.bam"

done

echo "All done."
date
```

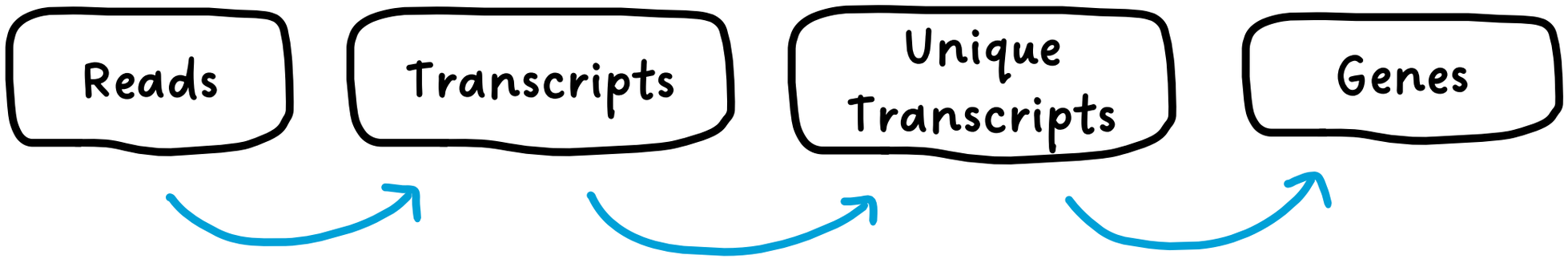
After running this script, we should have one sorted BAM file for each sample in our output directory.

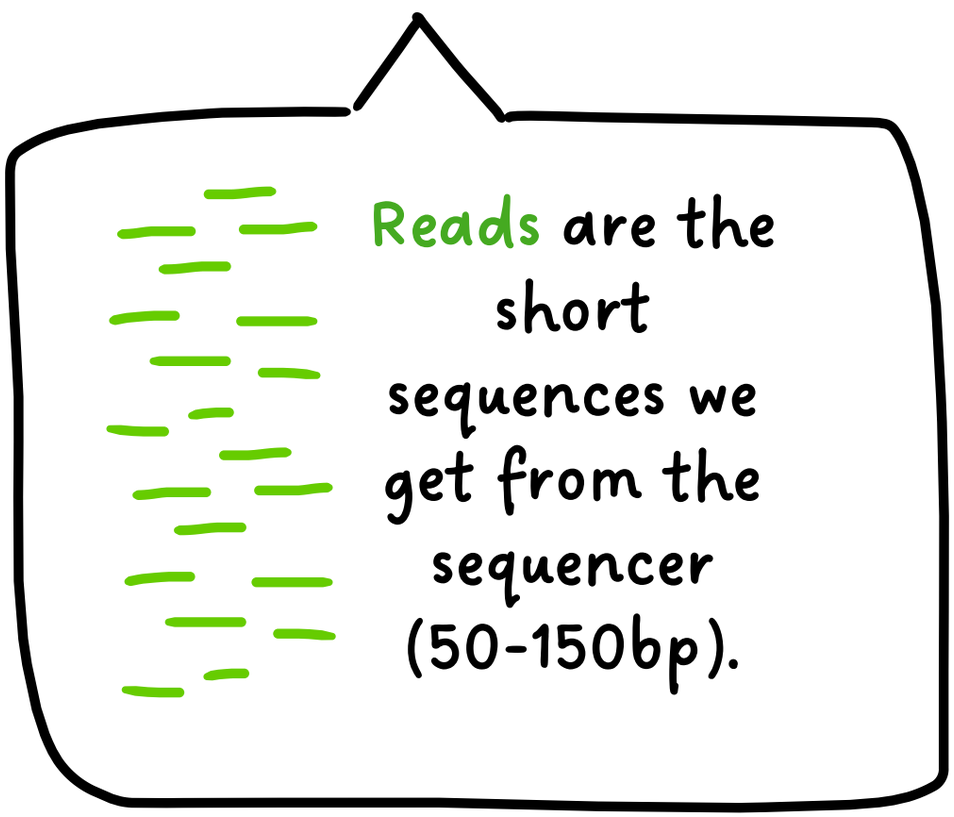
Now we can quantify with StringTie!

Quantifying Genes & Transcripts

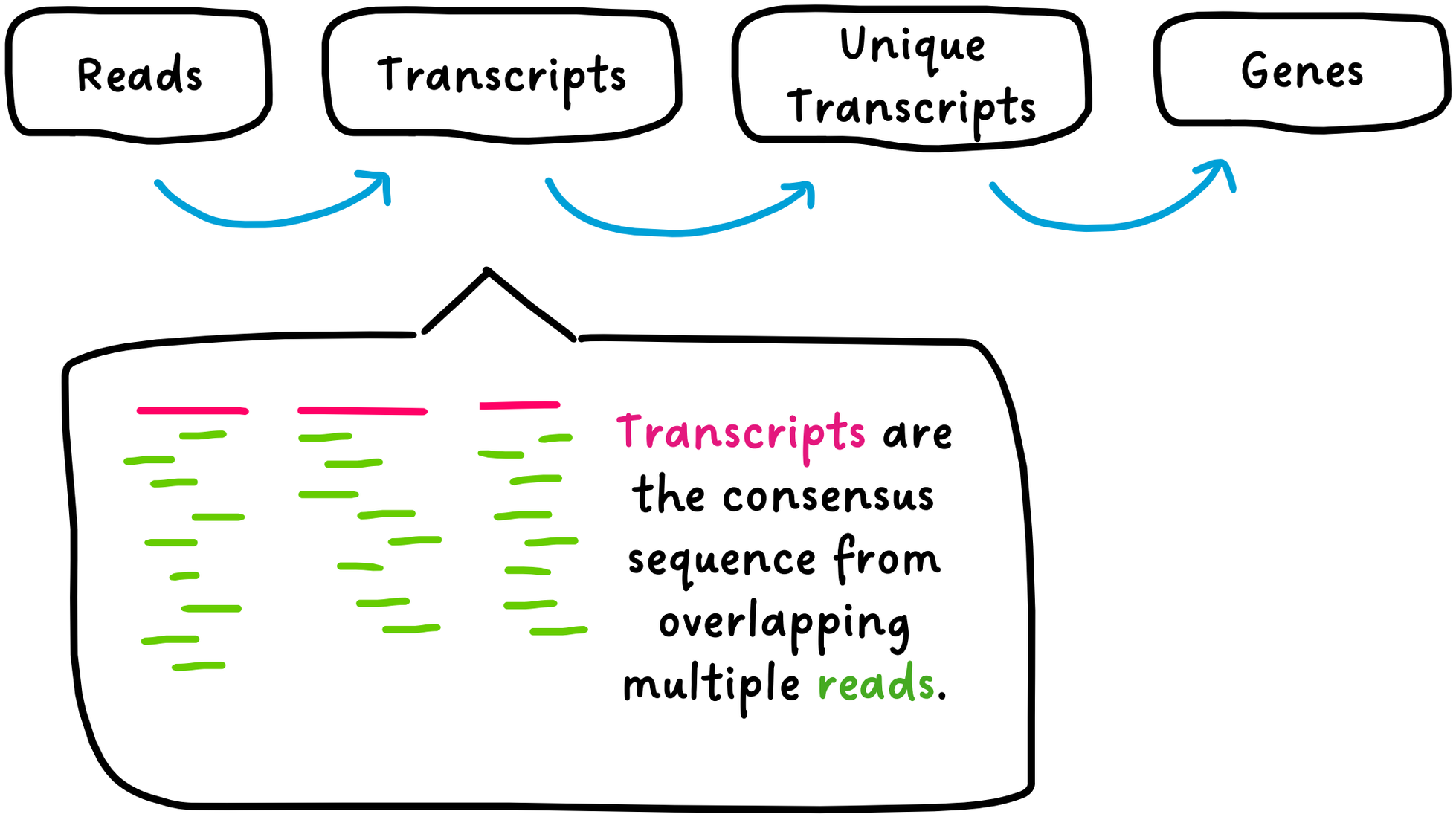
- What is the difference between quantifying genes vs transcripts?
- How do I quantify my aligned reads?

Quantifying Genes vs Transcripts

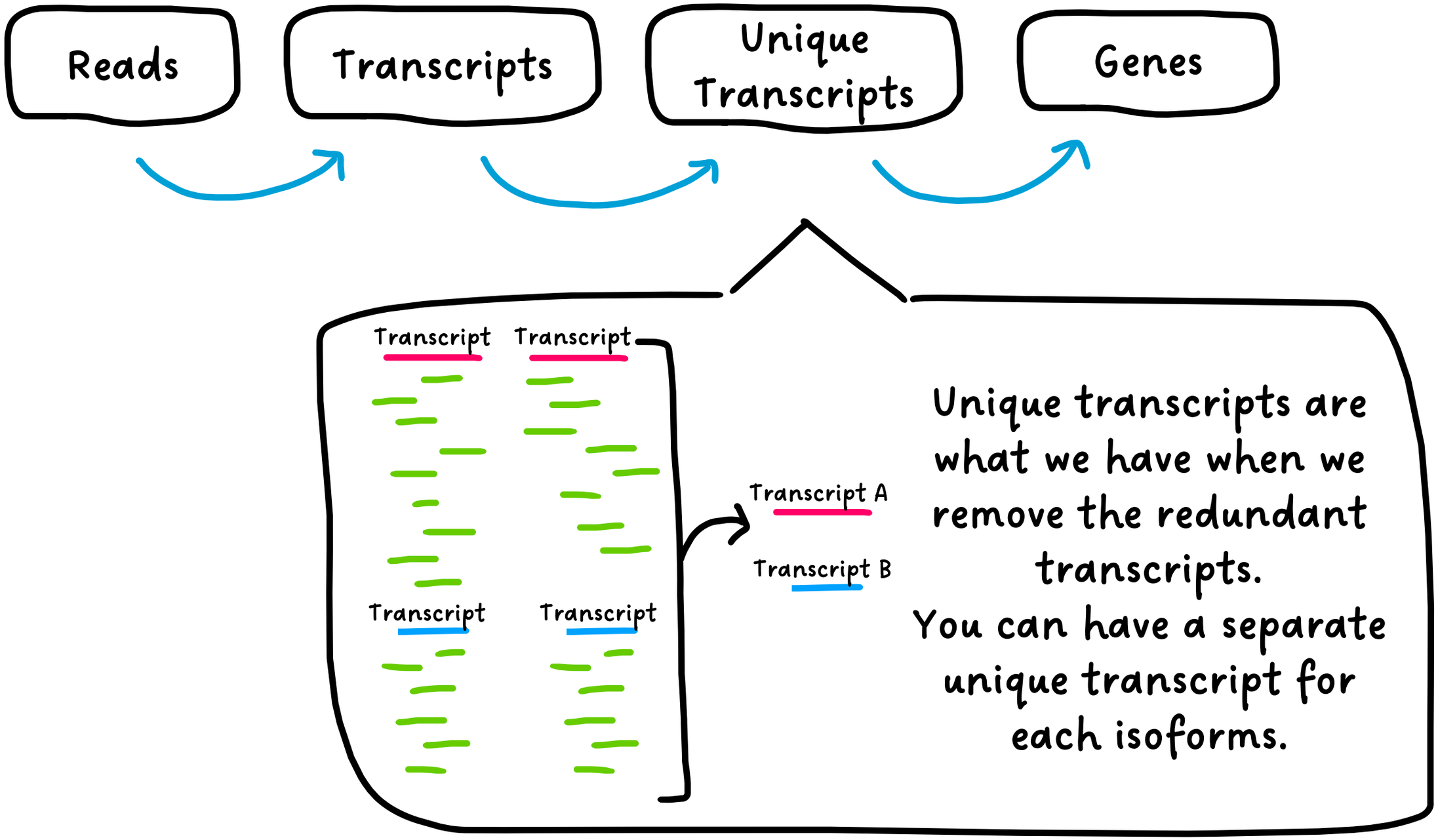




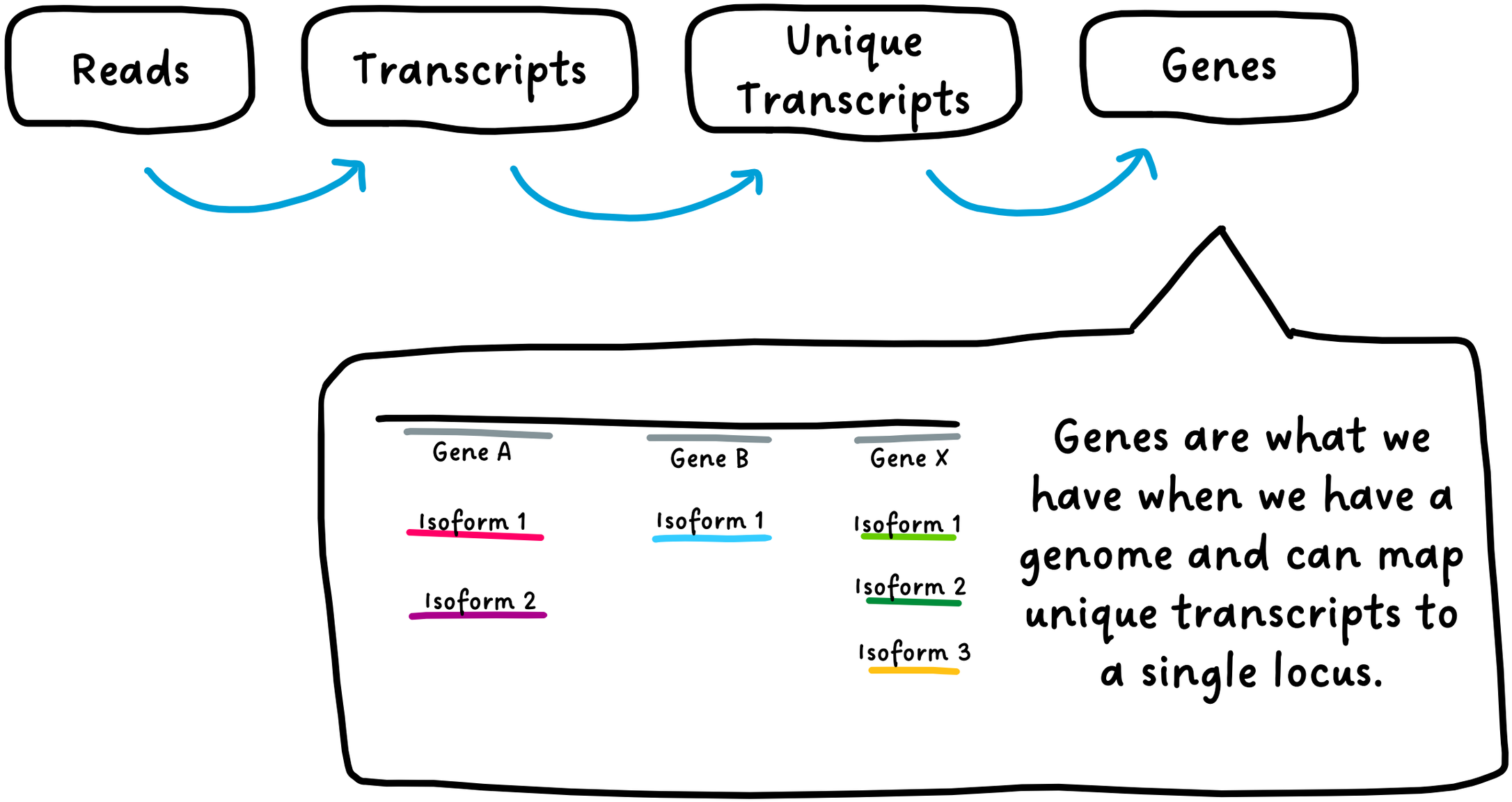
Quantifying Genes vs Transcripts



Quantifying Genes vs Transcripts



Quantifying Genes vs Transcripts



How do I quantify my aligned reads?

If you used **HISAT2** to align, then you use **StringTie** to quantify.

- StringTie uses reads already aligned to genomic coordinates.

If you used **Trinity** to assemble, then you use **RSEM** to quantify.

- RSEM quantifies transcripts directly and expects a transcript reference (although it can technically use a genome reference too).

Both will give you a final output of a counts matrix that you can use for DE analysis.

https://github.com/gpertea/stringtie
https://github.com/deweylab/RSEM

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

→

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

→

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

1. Start by preparing the reference:

```
> rsem-prepare-reference --bowtie2 Trinity.fasta TrinityRef1
```

This is the type of index you want to create

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

→

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

→

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

1. Start by preparing the reference:

```
> rsem-prepare-reference --bowtie2 Trinity.fasta TrinityRef1
```

This is the output from Trinity (de novo assembly)

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

1. Start by preparing the reference:

```
> rsem-prepare-reference --bowtie2 Trinity.fasta TrinityRef1
```

This is the output

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

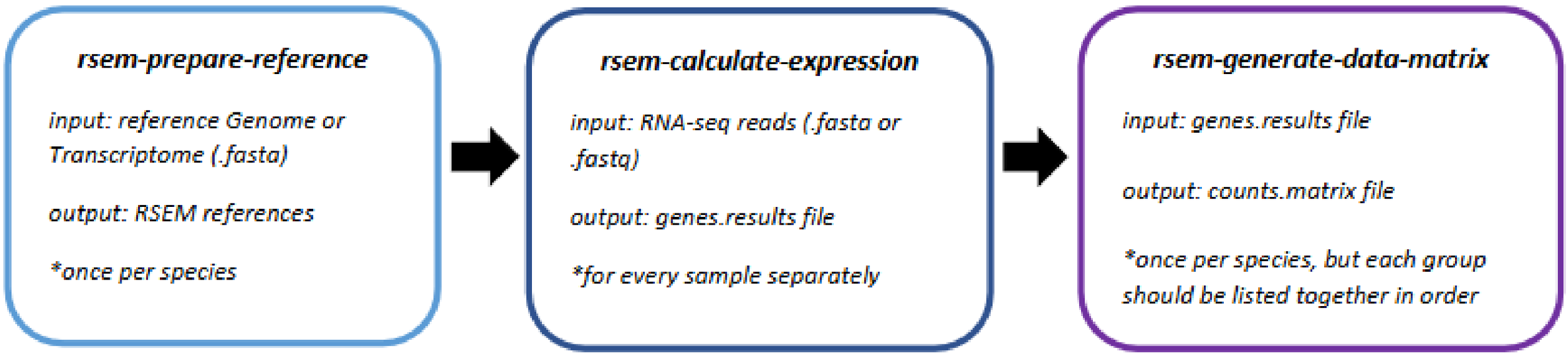
2. Then you will calculate the expression levels using the reference:

```
> rsem-calculate-expression --paired-end sample_R1.fastq sample_R2.fastq \
```

TrinityRef1 Male1

Here you would input your trimmed paired read files

How do I quantify my aligned reads with RSEM?



2. Then you will calculate the expression levels using the reference:

```
> rsem-calculate-expression --paired-end sample_R1.fastq sample_R2.fastq \
TrinityRef1 Sample1
```

Here is the reference you previously generated

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

2. Then you will calculate the expression levels using the reference:

```
> rsem-calculate-expression --paired-end sample_R1.fastq sample_R2.fastq \
TrinityRef1 Sample1
```

Sample name, this is the basename and RSEM will generate output files using this (Sample1.genes.results)

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

**once per species, but each group should be listed together in order*

2. Then you will calculate the expression levels using the reference:

```
> rsem-generate-matrix Sample1.genes.results Sample2.genes.results >
combinedSamples_counts.matrix
```

These are all the expression output files for all your samples

How do I quantify my aligned reads with RSEM?

rsem-prepare-reference

input: reference Genome or Transcriptome (.fasta)

output: RSEM references

**once per species*

rsem-calculate-expression

input: RNA-seq reads (.fasta or .fastq)

output: genes.results file

**for every sample separately*

rsem-generate-data-matrix

input: genes.results file

output: counts.matrix file

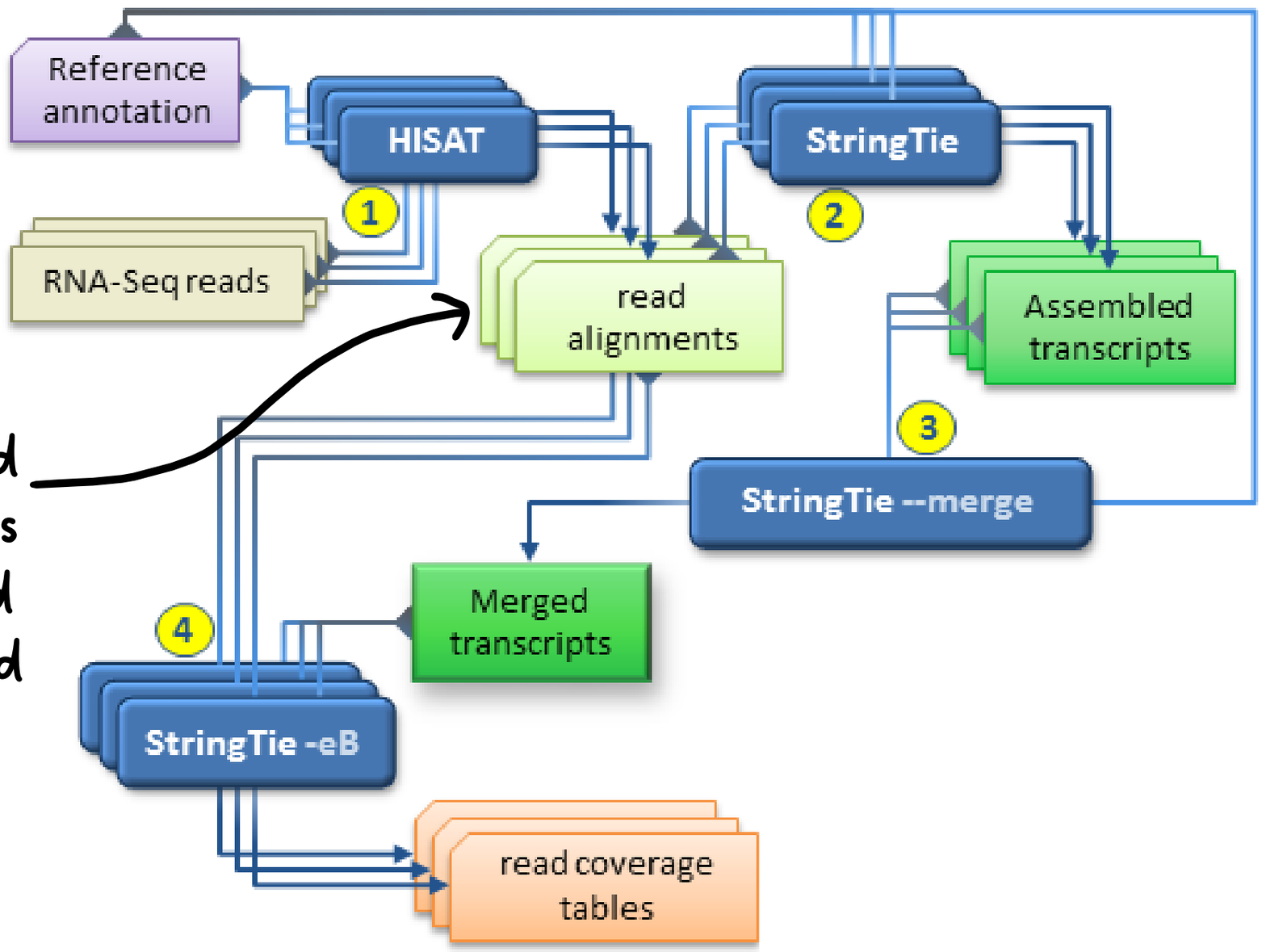
**once per species, but each group should be listed together in order*

2. Then you will calculate the expression levels using the reference:

> rsem-generate-matrix Sample1.genes.results Sample2.genes.results > combinedSamples_counts.matrix

Here is the output that will generate a matrix that combines all your samples together

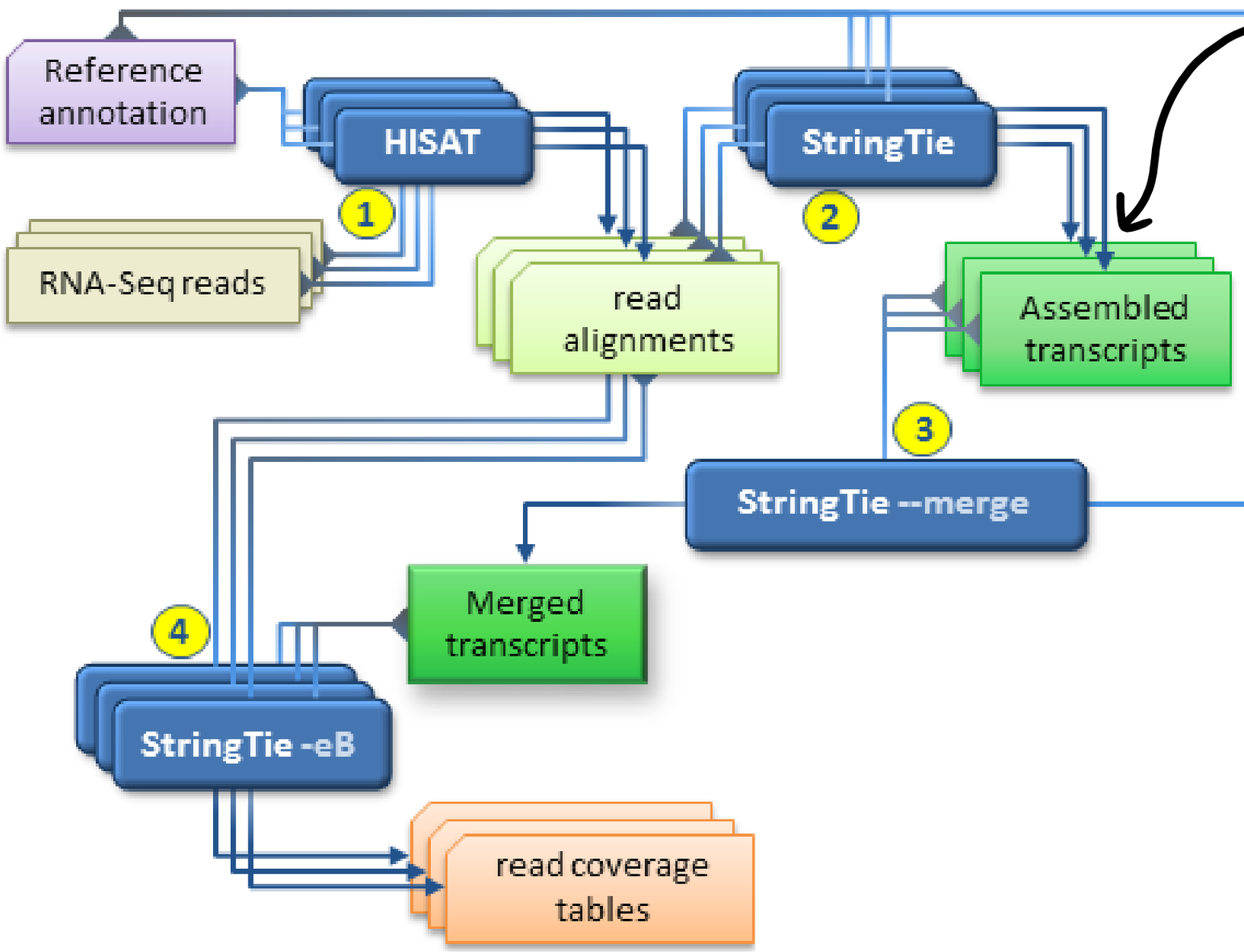
How do I quantify my aligned reads with StringTie?



1. We took our trimmed paired reads and aligned them (sorted BAM files).

https://ccb.jhu.edu/software/stringtie/index.shtml?t=manual

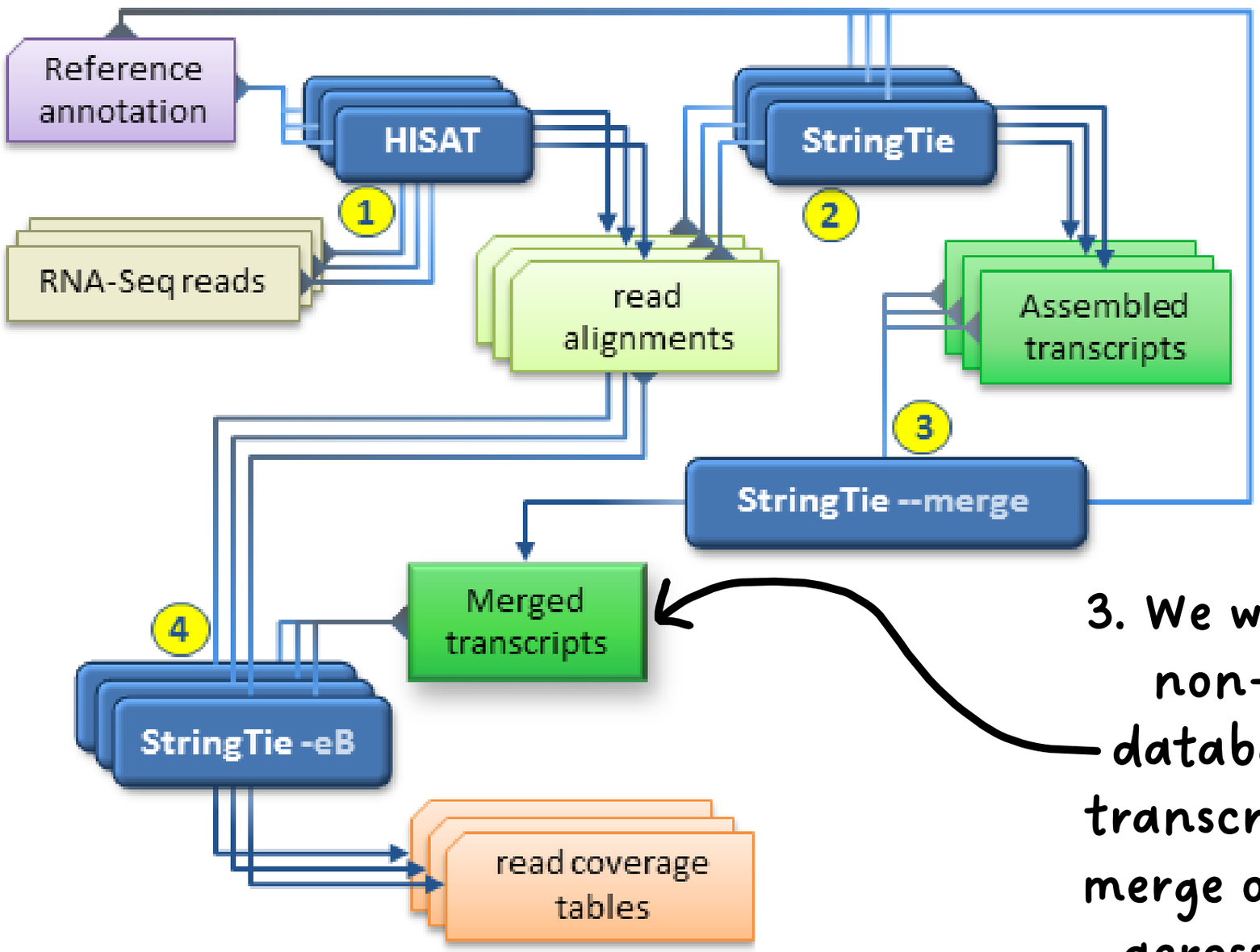
How do I quantify my aligned reads with StringTie?



2. We will take these aligned reads and assemble them into transcripts.

https://ccb.jhu.edu/software/stringtie/index.shtml?t=manual

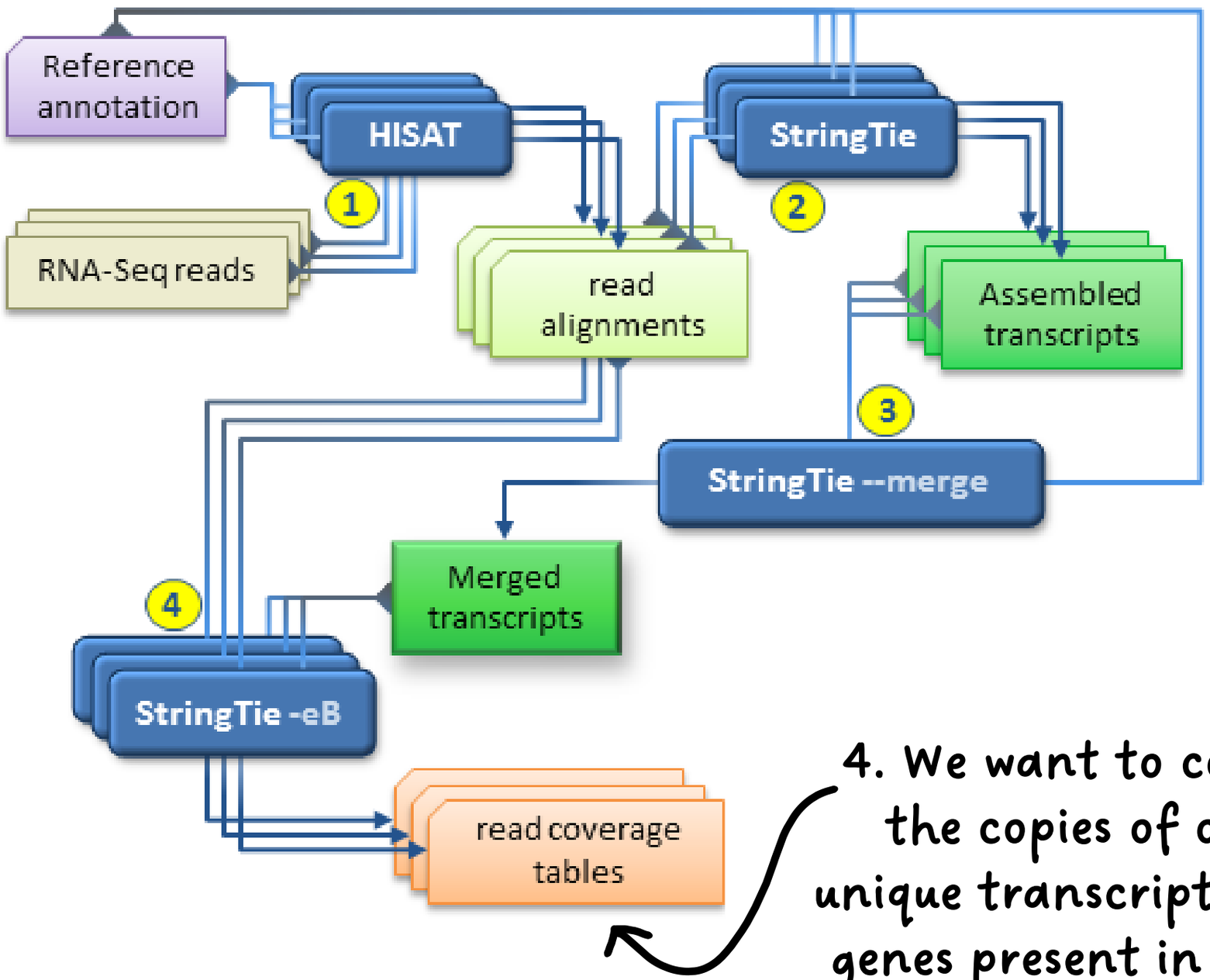
How do I quantify my aligned reads with StringTie?



3. We want to form a non-redundant database of unique transcripts, so we will merge our transcripts across all samples.

https://ccb.jhu.edu/software/stringtie/index.shtml?t=manual

How do I quantify my aligned reads with StringTie?



4. We want to count the copies of our unique transcripts and genes present in each sample.

https://ccb.jhu.edu/software/stringtie/index.shtml?t=manual

Running StringTie

1. The input for StringTie is our sorted BAM files. Then we start by running a command to assemble the transcripts.

```
> stringtie sample1_sorted.bam \
-o sample1_gtf.gtf \
-G annotation.gff \
-A sample1_abund.tab \
-C sample1_cov_refs.gtf
```

This is our sorted BAM file (this is the main input).

https://ccb.jhu.edu/software/stringtie/

Running StringTie

1. The input for StringTie is our sorted BAM files. Then we start by running a command to assemble the transcripts.

```
> stringtie sample1_sorted.bam \
-o sample1_gtf.gtf \
-G annotation.gff \
-A sample1_abund.tab \
-C sample1_cov_refs.gtf
```

-o is for the main output, which is a file that has assembled transcripts

https://ccb.jhu.edu/software/stringtie/

Running StringTie

1. The input for StringTie is our sorted BAM files. Then we start by running a command to assemble the transcripts.

```
> stringtie sample1_sorted.bam \
-o sample1_gtf.gtf \
-G annotation.gff \
-A sample1_abund.tab \
-C sample1_cov_refs.gtf
```

-G contains the genome annotation (this is optional, you don't need to have an annotation, but it's good to use if you have one)

https://ccb.jhu.edu/software/stringtie/

Running StringTie

1. The input for StringTie is our sorted BAM files. Then we start by running a command to assemble the transcripts.

```
> stringtie sample1_sorted.bam \
-o sample1_gtf.gtf \
-G annotation.gff \
-A sample1_abund.tab \
-C sample1_cov_refs.gtf
```

-A is for the abundance output file

https://ccb.jhu.edu/software/stringtie/

Running StringTie

1. The input for StringTie is our sorted BAM files. Then we start by running a command to assemble the transcripts.

```
> stringtie sample1_sorted.bam \
-o sample1_gtf.gtf \
-G annotation.gff \
-A sample1_abund.tab \
-C sample1_cov_refs.gtf
```

-C if you have an annotation, you can create this output file, which links the transcripts with the names in the annotation

https://ccb.jhu.edu/software/stringtie/

Running StringTie

2. Then we take the assembled transcripts, and we merge them all together to generate a non-redundant uniform transcriptome.

```
> stringtie --merge \
-o merged_transcripts.gtf \
-G annotation.gff \
sample*.gtf
```

-o this is the output that contains the non-redundant uniform transcriptome from all our samples

https://ccb.jhu.edu/software/stringtie/

Running StringTie

2. Then we take the assembled transcripts, and we merge them all together to generate a non-redundant uniform transcriptome.

```
> stringtie --merge \
-o merged_transcripts.gtf \
-G annotation.gff \
sample*.gtf
```

-G contains the genome annotation (this is optional, you don't need to have an annotation)

https://ccb.jhu.edu/software/stringtie/

Running StringTie

2. Then we take the assembled transcripts, and we merge them all together to generate a non-redundant uniform transcriptome.

```
> stringtie --merge \
-o merged_transcripts.gtf \
-G annotation.gff \
sample*.gtf
```

this is the output gtf from ALL our samples (from the previous StringTie command)

these will all be combined in this step

https://ccb.jhu.edu/software/stringtie/

Running StringTie

2. Then we take the assembled transcripts, and we merge them all together to generate a non-redundant uniform transcriptome.

```
> stringtie --merge \
-o merged_transcripts.gtf \
-G annotation.gff \
sample1.gtf \
sample2.gtf \
sample3.gtf \
sample4.gtf \
sample5.gtf
```

For example, it might look something like this

https://ccb.jhu.edu/software/stringtie/

Running StringTie

3. Then we re-run StringTie on our non-redundant merged transcriptome, to get counts for all our non-redundant transcripts.

```
> stringtie sample1_sorted.bam \
-G merged_transcripts.gtf \
-o sample1_merged_gtf.gtf \
-A sample1 _merged_ abund.tab
```

this is our original input files

https://ccb.jhu.edu/software/stringtie/

Running StringTie

3. Then we re-run StringTie on our non-redundant merged transcriptome, to get counts for all our non-redundant transcripts.

```
> stringtie sample1_sorted.bam \
-G merged_transcripts.gtf \
-o sample1_merged_gtf.gtf \
-A sample1 _merged_ abund.tab
```

-G this is NOT the genome annotation!
this is our newly generated non-redundant uniform transcriptome

https://ccb.jhu.edu/software/stringtie/

Running StringTie

3. Then we re-run StringTie on our non-redundant merged transcriptome, to get counts for all our non-redundant transcripts.

```
> stringtie sample1_sorted.bam \
-G merged_transcripts.gtf \
-o sample1_merged_gtf.gtf \
-A sample1 _merged_ abund.tab
```

-o and -A
there are our new outputs, similar to the first time but build from the non-redundant transcriptome

https://ccb.jhu.edu/software/stringtie/

Running StringTie

Let's give it a try! (Script is also on GitHub)

```
# Variables ----

# Define paths to programs, input, and output
# Can call program from shared directory
STRINGTIE_DIR="/home/workshop/stringtie"
GENOME_DIR="/home/workshop/genome"
ANNOTATION_GFF="$GENOME_DIR/genomic.gff"

INPUT_DIR="/home/ubuntu/04_sortedbam"
OUTPUT_DIR="/home/ubuntu/05_stringtie"
LOG_DIR="/home/ubuntu/05_stringtie/logs"

# Set number of threads
THREADS=3

# Create output and log directories ----
mkdir -p "$OUTPUT_DIR" "$LOG_DIR"
```

Let's start by defining our variables to store information about our paths.

We have a StringTie directory and a genome directory in our shared folder.

Running StringTie

Let's give it a try! (Script is also on GitHub)

```
# Variables ----

# Define paths to programs, input, and output
# Can call program from shared directory
STRINGTIE_DIR="/home/workshop/stringtie"
GENOME_DIR="/home/workshop/genome"
ANNOTATION_GFF="$GENOME_DIR/genomic.gff"

INPUT_DIR="/home/ubuntu/04_sortedbam"
OUTPUT_DIR="/home/ubuntu/05_stringtie"
LOG_DIR="/home/ubuntu/05_stringtie/logs"

# Set number of threads
THREADS=3

# Create output and log directories ----
mkdir -p "$OUTPUT_DIR" "$LOG_DIR"
```

Let's start by defining our variables to store information about our paths.

Then we want to define our inputs and outputs.

Running StringTie

Let's give it a try! (Script is also on GitHub)

```
# Variables ----

# Define paths to programs, input, and output
# Can call program from shared directory
STRINGTIE_DIR="/home/workshop/stringtie"
GENOME_DIR="/home/workshop/genome"
ANNOTATION_GFF="$GENOME_DIR/genomic.gff"

INPUT_DIR="/home/ubuntu/04_sortedbam"
OUTPUT_DIR="/home/ubuntu/05_stringtie"
LOG_DIR="/home/ubuntu/05_stringtie/logs"

# Set number of threads
THREADS=3

# Create output and log directories ----
mkdir -p "$OUTPUT_DIR" "$LOG_DIR"
```

Let's start by defining our variables to store information about our paths.

Then we want to define the computer capacity that StringTie can use.

For our class we have a limited number of threads

Running StringTie

Let's give it a try! (Script is also on GitHub)

```
# Variables ----

# Define paths to programs, input, and output
# Can call program from shared directory
STRINGTIE_DIR="/home/workshop/stringtie"
GENOME_DIR="/home/workshop/genome"
ANNOTATION_GFF="$GENOME_DIR/genomic.gff"

INPUT_DIR="/home/ubuntu/04_sortedbam"
OUTPUT_DIR="/home/ubuntu/05_stringtie"
LOG_DIR="/home/ubuntu/05_stringtie/logs"

# Set number of threads
THREADS=3

# Create output and log directories ----
mkdir -p "$OUTPUT_DIR" "$LOG_DIR"
```

Let's start by defining our variables to store information about our paths.

Then we want to make sure we create any directories we expect to use.

Running StringTie

Now, we are going to learn a new script writing technique, functions!

Since we have three steps in StringTie, it will save us time to have these written as three functions. We can feed the output of one into an input of another.

Running StringTie

For this, we are going to learn a new script writing technique, functions!

```bash
# Bash functions for each part of the StringTie pipeline ----

run_stringtie_before_merge() {
    echo "Starting StringTie before merge..."

    for BAM in "$INPUT_DIR"/*.sorted.bam
    do
        BASENAME=$(basename "$BAM" .sorted.bam)

        echo "Processing $BASENAME..."

        "$STRINGTIE_DIR/stringtie" "$BAM" \
            -o "$OUTPUT_DIR/$BASENAME.gtf" \
            -G "$ANNOTATION_GFF" \
            -A "$OUTPUT_DIR/$BASENAME.abund.tab" \
            -C "$OUTPUT_DIR/$BASENAME.cov_refs.gtf" \
            -p "$THREADS" \
            &>> "$LOG_DIR/$BASENAME.firstpass.log"
    done

    echo "StringTie before merge complete."
        date
}
```

Here we can define a function with:

```bash
function() {
        command
}
```

Running StringTie

For this, we are going to learn a new script writing technique, functions!

```
# Bash functions for each part of the StringTie pipeline ----

run_stringtie_before_merge() {
    echo "Starting StringTie before merge..."

    for BAM in "$INPUT_DIR"/*.sorted.bam
    do
        BASENAME=$(basename "$BAM" .sorted.bam)

        echo "Processing $BASENAME..."

        "$STRINGTIE_DIR/stringtie" "$BAM" \
            -o "$OUTPUT_DIR/$BASENAME.gtf" \
            -G "$ANNOTATION_GFF" \
            -A "$OUTPUT_DIR/$BASENAME.abund.tab" \
            -C "$OUTPUT_DIR/$BASENAME.cov_refs.gtf" \
            -p "$THREADS" \
            &>> "$LOG_DIR/$BASENAME.firstpass.log"
    done

    echo "StringTie before merge complete."
        date
}
```

Here we have another for loop, where we are running each of our samples through the StringTie command

Running StringTie

For this, we are going to learn a new script writing technique, functions!

```bash
# Bash functions for each part of the StringTie pipeline ----

run_stringtie_before_merge() {
    echo "Starting StringTie before merge..."

    for BAM in "$INPUT_DIR"/*.sorted.bam
    do
        BASENAME=$(basename "$BAM" .sorted.bam)

        echo "Processing $BASENAME..."

        "$STRINGTIE_DIR/stringtie" "$BAM" \
            -o "$OUTPUT_DIR/$BASENAME.gtf" \
            -G "$ANNOTATION_GFF" \
            -A "$OUTPUT_DIR/$BASENAME.abund.tab" \
            -C "$OUTPUT_DIR/$BASENAME.cov_refs.gtf" \
            -p "$THREADS" \
            &>> "$LOG_DIR/$BASENAME.firstpass.log"
    done

    echo "StringTie before merge complete."
        date
}
```

Then we are using a bunch of variables previous defined to run the first StringTie command.

Running StringTie

For this, we are going to learn a new script writing technique, functions!

```bash
# Bash functions for each part of the StringTie pipeline ----

run_stringtie_before_merge() {
    echo "Starting StringTie before merge..."

    for BAM in "$INPUT_DIR"/*.sorted.bam
    do
        BASENAME=$(basename "$BAM" .sorted.bam)

        echo "Processing $BASENAME..."

        "$STRINGTIE_DIR/stringtie" "$BAM" \
            -o "$OUTPUT_DIR/$BASENAME.gtf" \
            -G "$ANNOTATION_GFF" \
            -A "$OUTPUT_DIR/$BASENAME.abund.tab" \
            -C "$OUTPUT_DIR/$BASENAME.cov_refs.gtf" \
            -p "$THREADS" \
            &>> "$LOG_DIR/$BASENAME.firstpass.log"
    done

    echo "StringTie before merge complete."
        date
}
```

Then if you have multiple functions (we want 3 for each part of StringTie), we can schedule them in another function

```bash
main() {
    run_stringtie_before_merge
    run_stringtie_merge
    run_stringtie_after_merge
}

main "$@"
```

Running StringTie

For this, we are going to learn a new script writing technique, functions!

```
# Bash functions for each part of the StringTie pipeline ----

run_stringtie_before_merge() {
    echo "Starting StringTie before merge..."

    for BAM in "$INPUT_DIR"/*.sorted.bam
    do
        BASENAME=$(basename "$BAM" .sorted.bam)

        echo "Processing $BASENAME..."

        "$STRINGTIE_DIR/stringtie" "$BAM" \
            -o "$OUTPUT_DIR/$BASENAME.gtf" \
            -G "$ANNOTATION_GFF" \
            -A "$OUTPUT_DIR/$BASENAME.abund.tab" \
            -C "$OUTPUT_DIR/$BASENAME.cov_refs.gtf" \
            -p "$THREADS" \
            &>> "$LOG_DIR/$BASENAME.firstpass.log"
    done

    echo "StringTie before merge complete."
        date
}
```

Then we can run the scheduler function with: main "$@"

```
main() {
    run_stringtie_before_merge
    run_stringtie_merge
    run_stringtie_after_merge
}

main "$@"
```

Normalization

- Why is it important to normalize RNA-seq counts?
- What are the different methods to normalize?

Why is it important to normalize?

There can be many technical aspects that can generate a distorted representation of quantity.

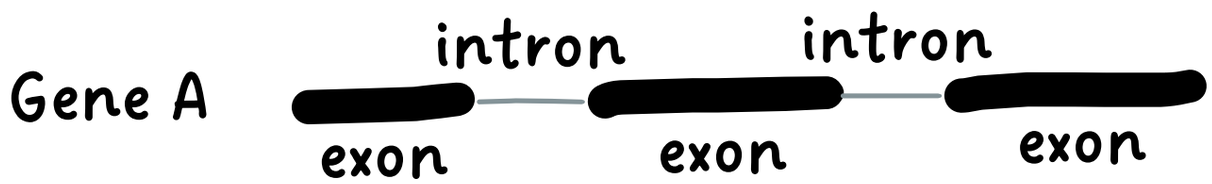
These technical aspects can generate distortions between samples and between genes.

Remember RNA-seq is a relative measurement not a total measurement!

Why is it important to normalize?

Differences between samples

Differences between genes



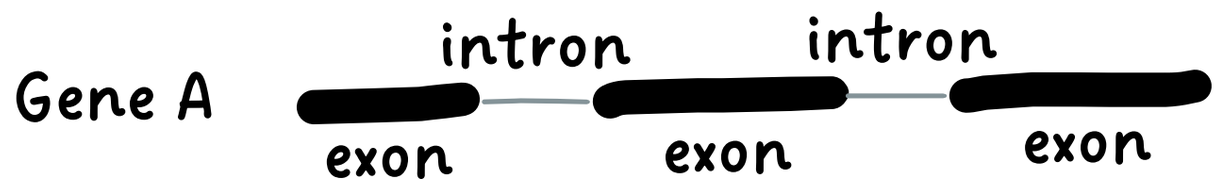
If Sample 1 has a total of 20M reads, and Sample 2 has a total of 40M reads sequenced, then for the same gene we see:

Garber et al. 2011

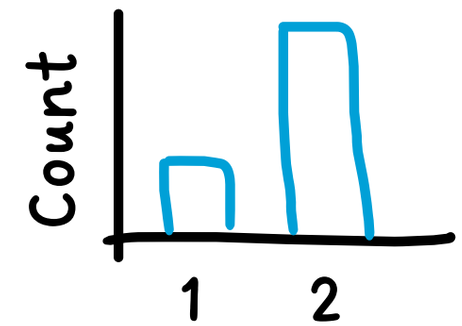
Why is it important to normalize?

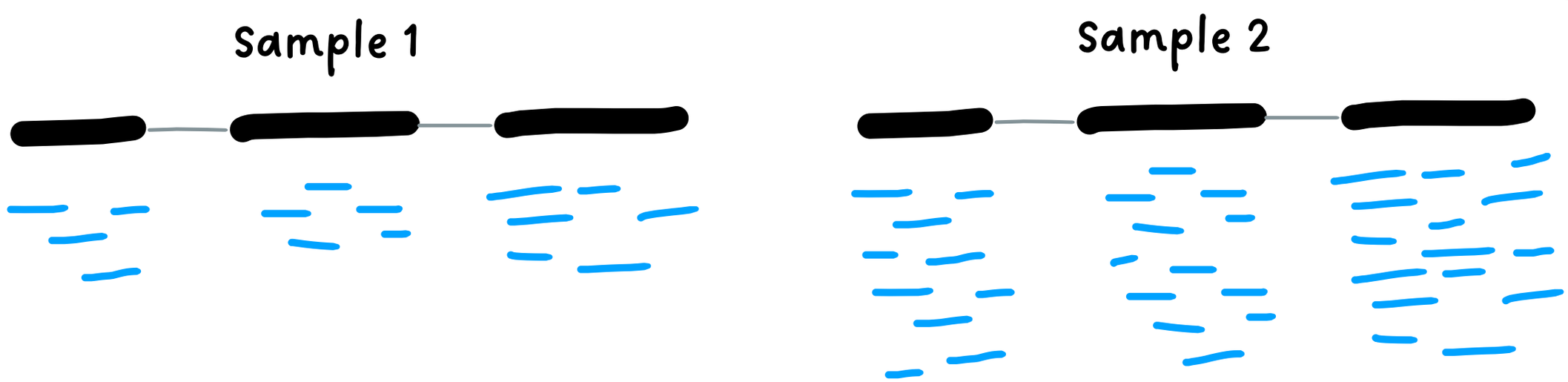
Differences between samples

Differences between genes



If Sample 1 has a total of 20M reads, and Sample 2 has a total of 40M reads sequenced, then for the same gene we see:





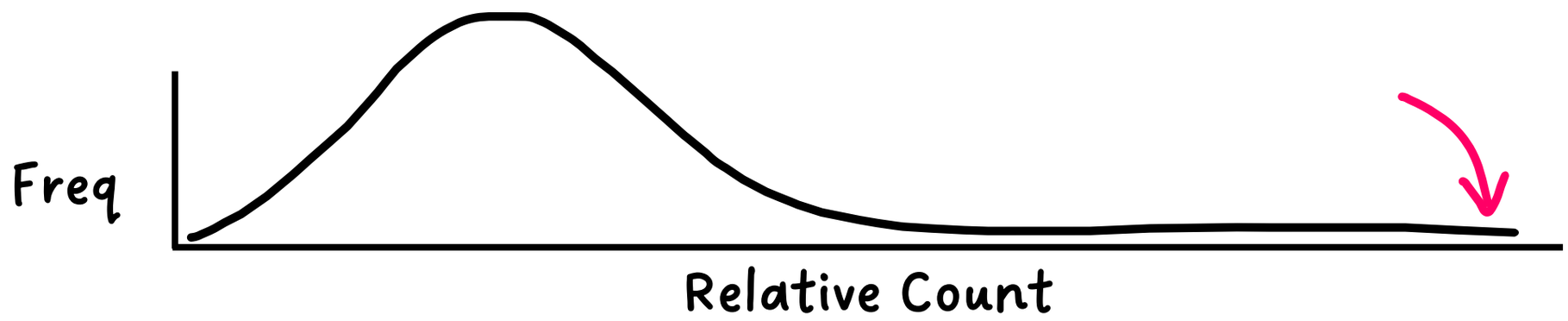
Garber et al. 2011

Why is it important to normalize?

Differences between samples

Differences between genes

Another issue can be compositional bias, where you have a couple genes with really high expression.

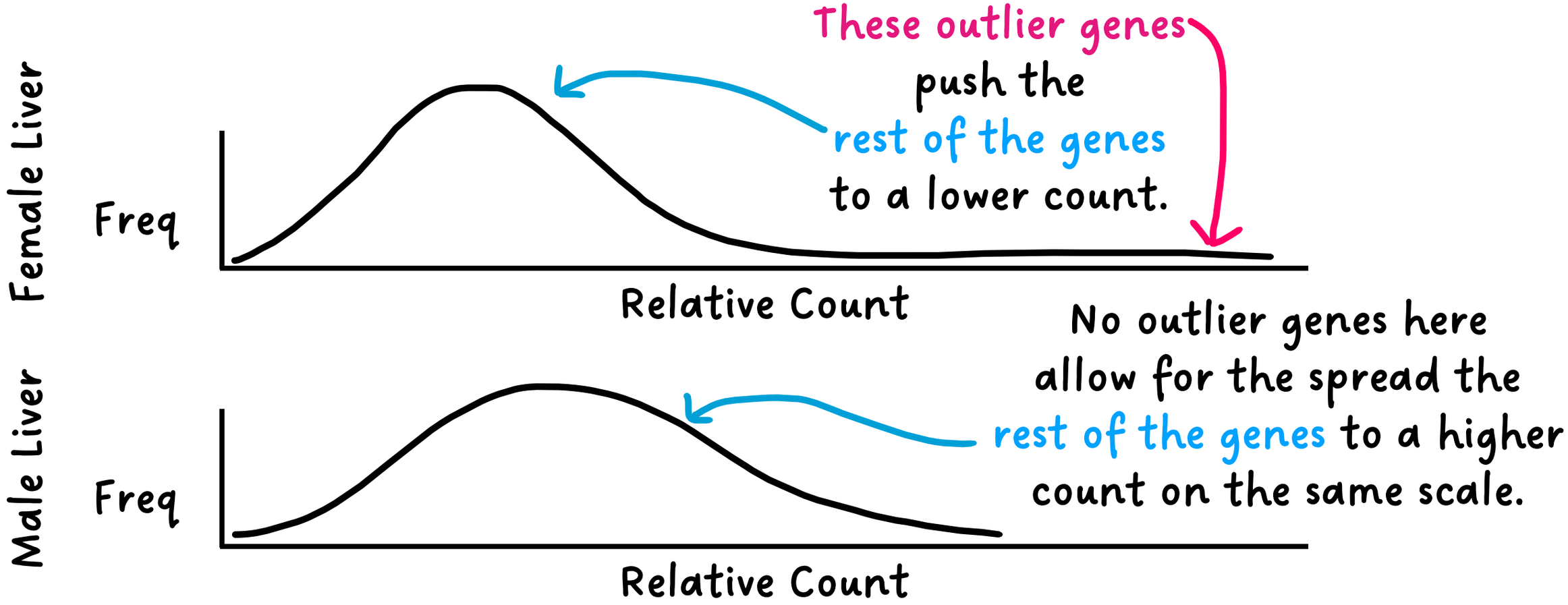


Why is it important to normalize?

Differences between samples

Differences between genes

Compositional bias can be different between samples
(such as when it's confounded to experimental groups).



Remember RNA-seq is a relative measurement!

Why is it important to normalize?

Differences between samples

Differences between genes

Gene A
(Short transcript)

Gene B
(Long transcript)

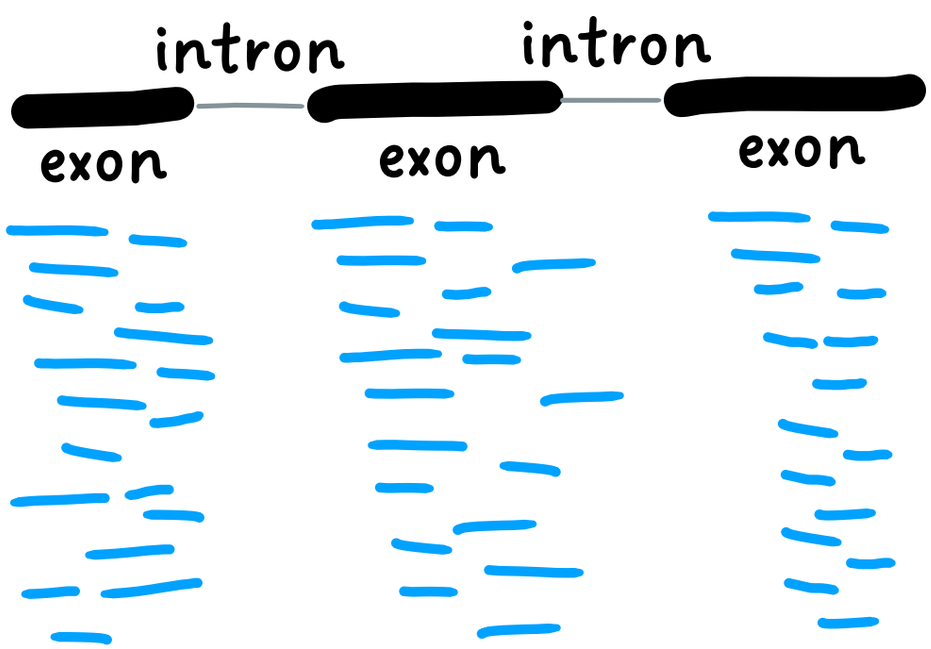
Garber et al. 2011

Why is it important to normalize?

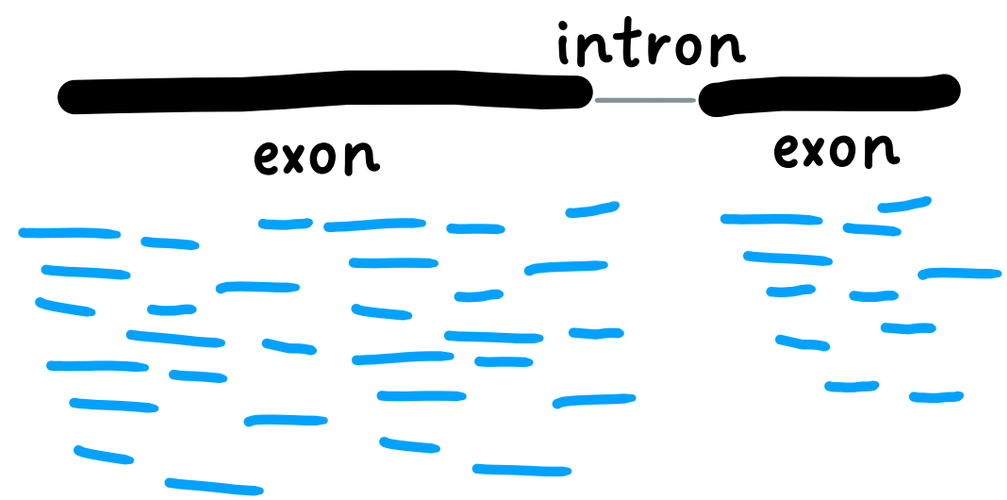
Differences between samples

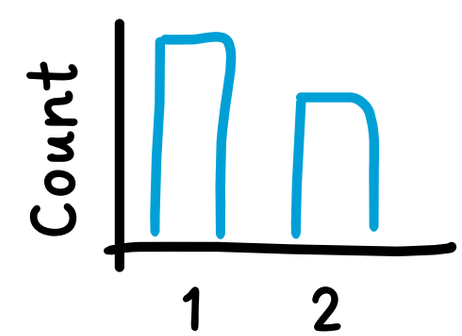
Differences between genes

Gene A
(Short transcript)



Gene B
(Long transcript)





If you have the same number of reads per gene,
then you will get more depth for shorter genes.

Garber et al. 2011

Why is it important to normalize?

Differences between samples

↓

This is okay to have IF you are comparing genes within a sample.

Differences between genes

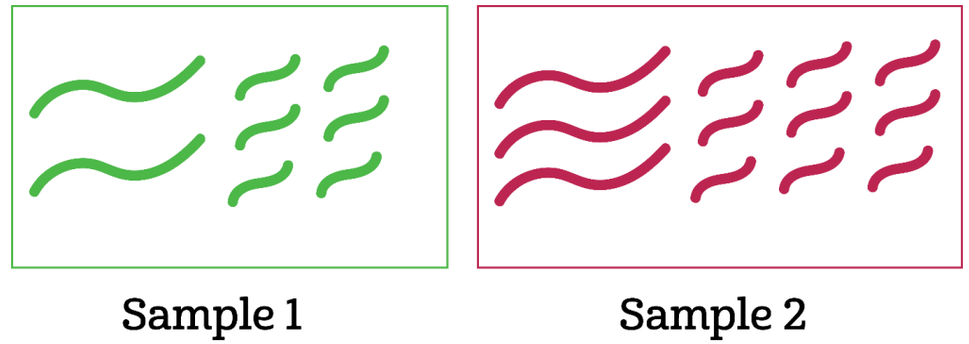
↓

This is okay to have IF we are comparing the same gene across samples (like a DE test).

Normalization methods

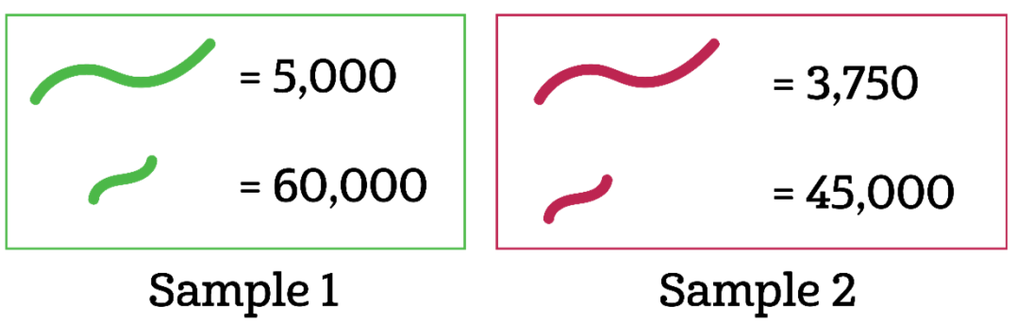
Method	Description	Accounted Factors	Usage
RPKM	**R**eads **P**er **K**ilobase per **M**illion reads mapped	• Sequencing depth • Gene length	• For single-end reads • NOT for between sample comparisons • Total expression value differs between samples • Outdated method
FPKM	**F**ragments **P**er **K**ilobase per **M**illion reads mapped	• Sequencing depth • Gene length	• For paired-end reads • NOT for between sample comparisons • Total expression value differs between samples • Outdated method
TPM	**T**ranscripts **P**er kilobase **M**illion	• Sequencing depth • Gene length	• For paired-end reads • Good for within sample comparisons • Good for between samples comparisons (assumes no compositional bias)
TMM	**T**rimmed **M**ean of **M**-values	• Sequencing depth • Compositional bias	• Good for between sample comparisons • NOT for within sample comparisons (no gene length adjustment) • Used for DE analysis in edgeR

Why FPKM is outdated



FPKM

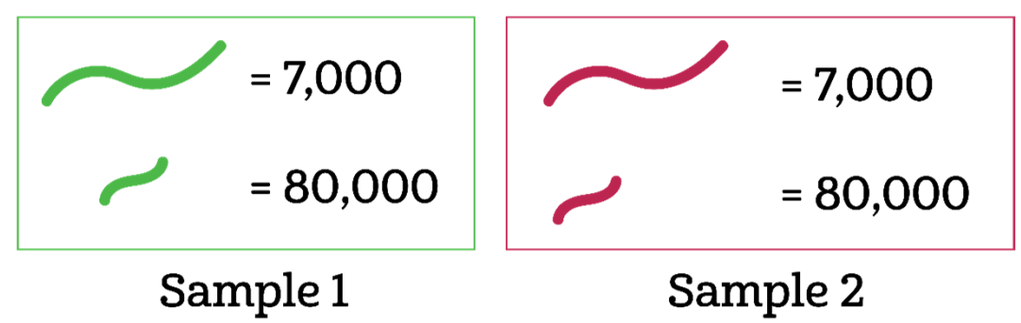
1. Normalize for sequence depth
2. Normalize for each gene's read count by length



= the sum of all FPKMs in each sample are not the same

TPM

1. Normalize for read length (=Reads per Kilobase)
2. Sum all RPK in a sample and divide by 1 million
3. Divide RPK value by a per M scaling factor



= the sum of all TPMs in each sample are the same

https://www.rna-seqblog.com/rpkm-fpkm-and-tpm-clearly-explained/

Normalization methods

Should I normalize my reads before inputting them into a differential expression program?

Normalization methods

Should I normalize my reads before inputting them into a differential expression program?

NO!

Why? Most differential expression programs already implement normalization strategies and expect raw counts.

Differential Expression Analysis

- What are the main differential expression (DE) programs?
- How does edgeR work?
- What are some factors that can affect a DE analysis?

Package	Model + Statistical Testing	Normalization Method	Usage
DESeq2 Love et al., 2014; *Genome Biology*	• Negative binomial GLM + • Wald test • Likelihood ratio test	• Median-of-ratios (RLE size factors)	• Dispersion estimates are shrunk towards a global trend, reducing variance in estimates • Shrinks log fold-change estimates using priors, this stabilizes estimates for low counts and small sample sizes • Stable effect sizes • Easier to use (good defaults)

Package	Model + Statistical Testing	Normalization Method	Usage
DESeq2 Love et al., 2014; *Genome Biology*	• Negative binomial GLM + • Wald test • Likelihood ratio test	• Median-of-ratios (RLE size factors)	• Dispersion estimates are shrunk towards a global trend, reducing variance in estimates • Shrinks log fold-change estimates using priors, this stabilizes estimates for low counts and small sample sizes • Stable effect sizes • Easier to use (good defaults)
EBSeq Leng et al., 2013; *Bioinformatics*	• Negative Binomial mixture + • Baysian posterior probability determines DE	• Median normalization / library size	• Does not fit a GLM per gene but fits a hierarchical mixture model. Gives a PPDE and posterior fold change per gene. • Specializes in differential isoform expression or multi-condition analyses • Performs better with more samples • Works well with transcript abundance estimation pipelines (RSEM)

Package	Model + Statistical Testing	Normalization Method	Usage
DESeq2 Love et al., 2014; *Genome Biology*	• Negative binomial GLM + • Wald test • Likelihood ratio test	• Median-of-ratios (RLE size factors)	• Dispersion estimates are shrunk towards a global trend, reducing variance in estimates • Shrinks log fold-change estimates using priors, this stabilizes estimates for low counts and small sample sizes • Stable effect sizes • Easier to use (good defaults)
EBSeq Leng et al., 2013; *Bioinformatics*	• Negative Binomial mixture + • Baysian posterior probability determines DE	• Median normalization / library size	• Does not fit a GLM per gene but fits a hierarchical mixture model. Gives a PPDE and posterior fold change per gene. • Specializes in differential isoform expression or multi-condition analyses • Performs better with more samples • Works well with transcript abundance estimation pipelines (RSEM)
edgeR Chen et al., 2025; *Nucleic Acids Research*	• Negative binomial GLM + • Exact test (2-group) • GLM likelihood ratio test • Quasi-likelihood F test	• TMM • Corrects composition bias	• Dispersion estimates are treated as uncertain, and handled through the quasi-likelihood modeling • Focuses on accurate dispersion estimation, making it robust even with small number of biological replicates (more statistical power) • Flexible in model implementation • Latest version can look at transcript-level differential expression (and even differential exon usage for alternative splicing events) • Can also implement an enrichment test for GO or KEGG pathway

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias
3. **Construct design matrix**: which samples are in which test group

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias
3. **Construct design matrix**: which samples are in which test group
4. **Estimate dispersions**: to quantify biological variability across replicates, (estimates common, trended, and tagwise dispersion)

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias
3. **Construct design matrix**: which samples are in which test group
4. **Estimate dispersions**: to quantify biological variability across replicates, (estimates common, trended, and tagwise dispersion)
5. **Fit quasi-likelihood GLM**: models gene expression while accounting for dispersion uncertainty

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias
3. **Construct design matrix**: which samples are in which test group
4. **Estimate dispersions**: to quantify biological variability across replicates, (estimates common, trended, and tagwise dispersion)
5. **Fit quasi-likelihood GLM**: models gene expression while accounting for dispersion uncertainty
6. **Hypothesis testing**: test whether expression differs between test groups

How does edgeR work?

1. **Filter low-count genes**: remove noisy low count genes, this improves the stability of variance estimates
2. **Normalize library sizes (TMM)**: correct compositional bias
3. **Construct design matrix**: which samples are in which test group
4. **Estimate dispersions**: to quantify biological variability across replicates, (estimates common, trended, and tagwise dispersion)
5. **Fit quasi-likelihood GLM**: models gene expression while accounting for dispersion uncertainty
6. **Hypothesis testing**: test whether expression differs between test groups
7. **Multiple testing correction**: Benjamini–Hochberg FDR to control for false discovery rate

What are some factors that can influence a DE test?

1. **Biological variability between replicates**: higher biological variability increases dispersion estimates, which reduces statistical power
2. **Low biological replication/sample size**: replicates improve statistical power by improving variance estimates
3. **Sequence depth**: low depth leads to more sampling noise and more genes with low counts (but having more than 20M reads gives diminishing returns as compared to increasing sample size)
4. **Analyzing low expression genes**: genes with low expression are harder to analyze because variance will be high and log fold changes become unstable
5. **Batch effects**: can produce false DE signals

Let’s check out StringTie results

(We are checking to make sure it ran, but we will use a complete gene count matrix!)

```
ubuntu@bioinformatics-1-do-not-delete:~/05_stringtie$ head gene_count_matrix.csv
gene_id,C4NPP,C7NPP,C7PP,C88NPP,C8PP,C9NPP,C9PP
MSTRG.3|ndufc1,3,4,0,0,0,0,0
MSTRG.1|LOC125980798,8,7,0,0,0,10,0
MSTRG.2|LOC125987402,0,0,0,0,0,0,0
MSTRG.4|hars,7,0,0,0,4,9,0
MSTRG.5|LOC137840633,0,0,0,0,0,0,0
gene-slc7a11|slc7a11,4,0,0,0,0,0,0
gene-LOC125981049|LOC125981049,0,0,0,0,0,0,0
gene-LOC137840640|LOC137840640,0,0,0,0,0,0,0
gene-LOC137840637|LOC137840637,0,0,0,0,0,0,0
```

We should see the gene and the count for each sample, separate by a comma. You can also open this csv file in Excel to make it easier to read.

Let’s check out StringTie results

(We are checking to make sure it ran, but we will use a complete gene count matrix!)

```
ubuntu@bioinformatics-1-do-not-delete:~/05_stringtie$ head gene_count_matrix.csv
gene_id,C4NPP,C7NPP,C7PP,C88NPP,C8PP,C9NPP,C9PP
MSTRG.3|ndufc1,3,4,0,0,0,0,0
MSTRG.1|LOC125980798,8,7,0,0,0,10,0
MSTRG.2|LOC125987402,0,0,0,0,0,0,0
MSTRG.4|hars,7,0,0,0,4,9,0
MSTRG.5|LOC137840633,0,0,0,0,0,0,0
gene-slc7a11|slc7a11,4,0,0,0,0,0,0
gene-LOC125981049|LOC125981049,0,0,0,0,0,0,0
gene-LOC137840640|LOC137840640,0,0,0,0,0,0,0
gene-LOC137840637|LOC137840637,0,0,0,0,0,0,0
```

How well labelled our data is, reflects how complete the annotation is.
You can see there are some genes “gene-LOC####” that do not have a real gene name but a genomic location.

Let’s check out StringTie results

(We are checking to make sure it ran, but we will use a complete gene count matrix!)

```
ubuntu@bioinformatics-1-do-not-delete:~/05_stringtie$ head gene_count_matrix.csv
gene_id,C4NPP,C7NPP,C7PP,C88NPP,C8PP,C9NPP,C9PP
MSTRG.3|ndufc1,3,4,0,0,0,0,0
MSTRG.1|LOC125980798,8,7,0,0,0,10,0
MSTRG.2|LOC125987402,0,0,0,0,0,0,0
MSTRG.4|hars,7,0,0,0,4,9,0
MSTRG.5|LOC137840633,0,0,0,0,0,0,0
gene-slc7a11|slc7a11,4,0,0,0,0,0,0
gene-LOC125981049|LOC125981049,0,0,0,0,0,0,0
gene-LOC137840640|LOC137840640,0,0,0,0,0,0,0
gene-LOC137840637|LOC137840637,0,0,0,0,0,0,0
```

If your species does not have an annotation, you would instead just see “MSTRG.#” and you would not see any gene names.

```
MSTRG.7100,14293,11145,11462,9558,10823,9095,9817,24719,28988,17402,11246,14971
MSTRG.9818,513,348,412,278,335,393,315,1037,948,354,406,425
MSTRG.15559,277,187,166,289,158,136,167,593,410,186,168,179
MSTRG.8960,2517,2578,2343,1885,1839,1636,1314,5064,4463,2381,1798,2889
MSTRG.20088,6,3,6,4,5,0,0,0,10,8,14,0
MSTRG.20089,0,13,0,12,10,0,13,15,19,8,30,0
```

Let’s take a look at our data

Our StringTie was run using truncated fastq files. This helps to keep run times short when working with practice file, but for the DE test, I generated a gene count matrix with real full-length data.

We will run edgeR (in R) on this gene count matrix and see what interesting DEGs emerge.

Let’s take a look at our data

We will be working with a dataset generated by running:
Trimmomatic > HISAT2 > StringTie

In the workshop directory you will have a counts matrix generated from a full dataset, that looks like this:

```
ubuntu@bioinformatics-1-do-not-delete:~/genecounts$ head gene_count_matrix.csv
gene_id,Control1_NP,Control2_NP,Control3_NP,Control4_NP,Control1_P,Control2_P,Control3_P,Control4_P
MSTRG.7100,14293,11145,11462,9558,10823,9095,9817,24719,28988,17402,11246,14971
MSTRG.9818,513,348,412,278,335,393,315,1037,948,354,406,425
MSTRG.15559,277,187,166,289,158,136,167,593,410,186,168,179
MSTRG.8960,2517,2578,2343,1885,1839,1636,1314,5064,4463,2381,1798,2889
MSTRG.20088,6,3,6,4,5,0,0,0,10,8,14,0
MSTRG.20089,0,13,0,12,10,0,13,15,19,8,30,0
MSTRG.20086,2863,3524,2722,2027,1935,1861,1906,5786,5788,3260,2516,2950
MSTRG.20087,0,0,0,0,0,0,0,0,17,6,16,0
MSTRG.20084,0,0,0,0,4,21,6,12,0,0,0,0
```

But how do we take these counts and have biologically meaningful results?

First, we need to know what the data is representing.

Let's take a look at our data

The data we will be working with is from Gulf pipefish.

© Fishes of Texas team, some rights reserved (CC-BY-SA)

Pipefish are relatives of seahorses, and seadragons.

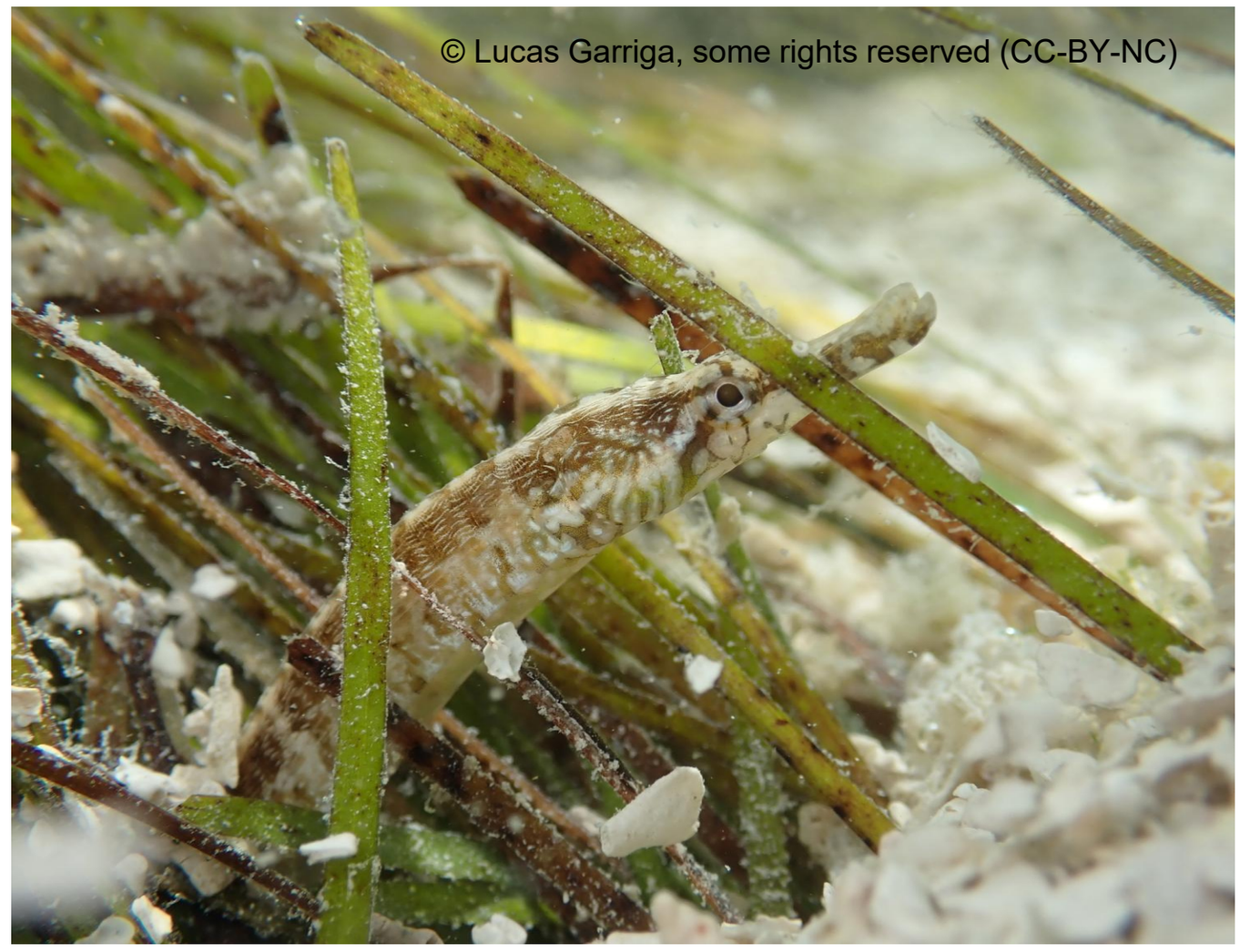
Similarly to seahorses, males carry the offspring in a complex structure called a brood pouch.

The male provisions a lot of resources during offspring development.

© Turner Brockman, some rights reserved (CC-BY-SA)

Let's take a look at our data

Pipefish predominately live in undisturbed seagrass beds.



These habitats are particularly sensitive to pollution, especially from waste treatment facilities.

One major concern is that environmental estrogens (from human contraceptives) will impair their ability to reproduce.

Let's take a look at our data

To study this concern, scientists have collected samples, and generated RNA-seq data of:

1. Control Pregnant Males
2. Control Non-Pregnant Males
3. Estrogen-exposed Pregnant Males

This gives us the ability to test multiple comparisons with a differential expression test.

Pregnant vs Not Pregnant
Control Pregnant vs EE Pregnant

Course Info

- Course Materials on Github: https://github.com/BernadetteBiology
- Explore bioinformatic tools (including mappers): https://australianbiocommons.github.io/toolfinder/
- How to install and run popular RNA-seq analysis programs (including StringTie and RSEM): https://bernadettebiology.weebly.com/protocols-tutorials--courses.html
- Photos, unless otherwise noted, are under © Bernadette J, some rights reserved (CC-BY-NC)

Further Reading

Baruzzo, G., Hayer, K., Kim, E. *et al.* Simulation-based comprehensive benchmarking of RNA-seq aligners. *Nat Methods* 14, 135–139 (2017).

Conesa A, Madrigal P, Tarazona S, Gomez-Cabrero D, Cervera A, McPherson A, Szcześniak MW, Gaffney DJ, Elo LL, Zhang X, et al. 2016. A survey of best practices for RNA-seq data analysis. Genome Biology 17: 1–19.

Garber et al. 2011. Computational methods for transcriptome annotation and quantification using RNA-seq. Nature Methods. 8:469-477.

Marinov et al. 2014. From single-cell to cell-pool transcriptomes: Stochasticity in gene expression and RNA splicing. Genome Research. 24:496–510.

Rapaport et al. 2013. Comprehensive evaluation of differential gene expression analysis methods for RNA-seq data. Genome Biology. 14:R95.

Seyednasrollah et al. 2013. Comparison of software packages for detecting differential expression in RNA-seq studies. Briefings in Bioinformatics.

Tarazona et al. 2011. Differential expression in RNA-seq: A matter of depth. Genome Res. 21: 2213-2223

https://www.labome.com/method/RNA-seq.html

http://deweylab.biostat.wisc.edu/rsem/

http://www.mi.fu-berlin.de/wiki/pub/ABI/GenomicsLecture12Materials/RNA-seq1.pdf

http://www.bioconductor.org/packages/release/bioc/vignettes/edgeR/inst/doc/edgeRUsersGuide.pdf